

hdfs commands

```
start-all.sh
hdfs dfs -ls /
hdfs dfs -mkdir /user
hdfs dfs -mkdir /user/bdalab
hdfs dfs -ls /user
nano localfile.txt
hdfs dfs -put localfile.txt /user/bdalab/
hdfs dfs -ls /user/bdalab/
hdfs dfs -cat /user/bdalab/localfile.txt
hdfs dfs -head /user/bdalab/localfile.txt
hdfs dfs -tail /user/bdalab/localfile.txt
hdfs dfs -cp /user/bdalab/localfile.txt /user/
bdalab/localfile_copy.txt
hdfs dfs -ls /user/bdalab/
hdfs dfs -get /user/bdalab/localfile.txt /home/
hdfs dfs -text /user/bdalab/localfile.txt
hdfs dfs -rm /user/bdalab/localfile_copy.txt
hdfs dfs -rm -r /user/bdalab
hdfs dfs -ls /user
```

Map reduce

Single node cluster :

Mapper.py

```
#!/usr/bin/env python3
import sys
import re
for line in sys.stdin:
    words = re.findall(r'\b\w+\b', line.lower())
    for word in words:
        print(f"{word}\t1")
reducer.py
#!/usr/bin/env python3
import sys
current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word, count = line.split('\t', 1)
    count = int(count)
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print(f"{current_word}\t{current_count}")
            current_count = count
            current_word = word
if current_word == word:
    print(f"{current_word}\t{current_count}")
```

```
hadoop@ubuntu22: ~$ chmod +x mapper.py reducer.py
nano input.txt
hadoop@ubuntu22: ~$ cat input.txt | ./mapper.py | sort -k1,1 | ./reducer.py
hadoop@ubuntu22: ~$ cat input.txt | ./mapper.py | sort | ./reducer.py
```

Multinode cluster + case study

```
nano mapper.py
#!/usr/bin/env python3
import sys
for line in sys.stdin:
    year, day, temp = line.strip().split()
    print(f"{year}\t{temp}")
nano reducer.py
#!/usr/bin/env python3
import sys
current_year = None
max_temp = -9999
for line in sys.stdin:
    line = line.strip()
    year, temp = line.split('\t')
    temp = int(temp)
    if current_year == year:
        if temp > max_temp:
            max_temp = temp
```

else:

```
    if current_year:
        print(f"{current_year}\t{max_temp}")
        current_year = year
        max_temp = temp
if current_year == year:
    print(f"{current_year}\t{max_temp}")
chmod +x mapper.py reducer.py
cat weather.txt | ./mapper.py | sort | ./reducer.py
nano weather.txt
hdfs dfs -mkdir /weather_input
hdfs dfs -put weather.txt /weather_input
hdfs dfs -ls /weather_input
ls /usr/lib/hadoop-mapreduce/hadoop-streaming.jar
find / -name "hadoop-streaming*.jar"
hadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar
hadoop jar /usr/lib/hadoop-mapreduce/hadoop-streaming.jar \ -mapper
mapper.py \ -reducer reducer.py \ -input /weather_input/weather.txt \ -
output /weather_output
hdfs dfs -ls /weather_output
hdfs dfs -cat /weather_output/part-00000
hdfs dfs -rm -r /weather_output
```

pig

Cleaner.py (Create this in VSCode / gedit / nano):

```
import re
import sys
for i in sys.stdin:
    print(re.sub(r'\s+', ' ', i))
Command: cat weather.txt | python3 cleaner.py > weather_cleaned.txt
Program:
records = LOAD 'weather_cleaned.txt' USING PigStorage(' ') AS
(number:int, temperature:int, time:CharArray, a:float);
dump records;
filtered_records = FILTER records BY temperature == 13910 AND (a <= 35
OR a >= 50);
dump filtered_records;
grouped_records = GROUP records BY time;
dump grouped_records;
max_temp = FOREACH grouped_records GENERATE group,
MAX(records.temperature);
dump max_temp;
describe records;
describe filtered_records;
describe grouped_records;
describe max_temp;
Weather.txt
~$ nano weather.txt
year maxtemp mintemp
2022 35 20
cleaner.py
~$ nano cleaner.py
import re
import sys
for i in sys.stdin:
    print(re.sub(r'\s+', ' ', i))
~$ cat weather.txt | python3 cleaner.py > weather_cleaned.txt
~$ pig -x local
grunt> weather_data = LOAD 'weather.txt' USING PigStorage(' ') AS
(year:int, maxtemp:int,
mintemp:int);
grunt> DUMP weather_data
grunt> filtered_weather = FILTER weather_data BY maxtemp > 35;
grunt> DUMP filtered_weather;
grunt> grouped_weather = GROUP weather_data BY year;
grunt> DUMP grouped_weather;
grunt> grouped_weather_count = FOREACH grouped_weather GENERATE
group AS year,
COUNT(weather_data) AS record_count;
grunt> DUMP grouped_weather_count;
```

Hive

```
start-all.sh
jps
hive
CREATE TABLE temperature_data (
full_date STRING,zip INT,temperature INT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ';';
SHOW TABLES; DESCRIBE temperature_data;
LOAD DATA LOCAL INPATH '/home/hadoop/temperature_data.txt'
OVERWRITE INTO TABLE temperature_data;
SELECT * FROM temperature_data;
SELECT full_date, MAX(temperature)
FROM temperature_data WHERE temperature > 29
GROUP BY full_date;
CREATE EXTERNAL TABLE ext_table (dummy STRING)
LOCATION '/home/hadoop/ext_table';
LOAD DATA LOCAL INPATH '/home/hadoop/data.txt'
INTO TABLE ext_table;
SELECT * FROM ext_table;
```

Scala :

```
cd ~/spark-3.5.2-bin-hadoop3/bin
./spark-shell
./pyspark
scala> val rdd=
spark.sparkContext.textFile("file:///home/hadoop/Desktop/Sparkcount.txt")
rdd: org.apache.spark.rdd.RDD[String] =
file:///home/hadoop/Desktop/Sparkcount.txt MapPartitionsRDD[1] at
textFile at
scala> rdd.collect()
res0: Array[String] = Array(Dear Bear River, Car Car River, Deer Car Bear,
Dear Deer River)
scala> val rdd1 = rdd.flatMap(x => x.split(" "))
scala> rdd1.collect()
scala> val rdd2 = rdd1.map( x => (x,1))
scala> val rdd2 = rdd1.map( x => (x,1))
rdd2: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[3] at
map at <console>:23
scala> rdd2.collect()
res2: Array[(String, Int)] = Array((Dear,1), (Bear,1), (River,1), (Car,1), (Car,1),
(River,1), (Deer,1), (Car,1), (Bear,1), (Dear,1), (Deer,1), (River,1))
scala> val rdd3 = rdd2.reduceByKey( (a,b) => a+b)
rdd3: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[4] at
reduceByKey at <console>:23
scala> rdd3.collect()
res3: Array[(String, Int)] = Array((Dear,2), (Deer,2), (Bear,2), (Car,3),
(River,3))
scala> rdd3.toDF("word", "count").show()
| word|count|
| Dear|  2|
| Deer|  2|
| Bear|  2|
| Car |  3|
| River|  3|
```

Pyspark :

```
cd ~/spark-3.5.2-bin-hadoop3/bin
./spark-shell
./pyspark
>>> input_data = sc.textFile('/home/hadoop/Desktop/Sparkcount.txt')
>>> word_counts = input_data.flatMap(lambda line: line.split(" "))
>>> word_counts1 = word_counts.map(lambda word: (word, 1))
>>> word_counts1.collect()
>>> final_counts = word_counts1.reduceByKey(lambda a,b: a+b)
>>> final_counts.collect()word_counts1 = word_counts.map(lambda word:
(word, 1))
final_counts = word_counts1.reduceByKey(lambda a, b: a + b)
final_counts.collect()
```

Spark Streaming :

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum
from pyspark.sql.types import StructType, StructField, IntegerType,
DoubleType
spark = SparkSession.builder \
    .appName("RetailStoreStreaming") \
    .config("spark.ui.showConsoleProgress", "false") \
    .getOrCreate()
spark.sparkContext.setLogLevel("ERROR")
schema = StructType([
    StructField("ProductID", IntegerType(), True),
    StructField("Amount", DoubleType(), True)])
input_path = "/home/hadoop/spark_input"
retail_stream_df = spark \
    .readStream \
    .format("csv") \
    .option("header", "true") \
    .schema(schema) \
    .load(input_path)
processed_df =
retail_stream_df.groupBy("ProductID").agg(sum("Amount")).alias("TotalSal
es"))
processed_df
=retail_stream_df.groupBy("ProductID").agg(sum("Amount")).alias("TotalSa
les"))
query = processed_df.writeStream \
    .outputMode("complete") \
    .format("console") \
    .start() query.awaitTermination()
hadoop@ubuntu22:~/spark-3.5.2-bin-hadoop3/bin$ ./spark-submit
/home/hadoop/sparkstream.py
```

Kafka

```
sudo apt update
sudo apt install openjdk-21-jdk -y
java -version
sudo useradd kafka -m
sudo passwd kafka
su - kafka
cd ~
wget https://archive.apache.org/dist/kafka/3.6.1/kafka_2.13-3.6.1.tgz
tar -xvzf kafka_2.13-3.6.1.tgz
mv kafka_2.13-3.6.1 kafka
cd kafka
bin/kafka-storage.sh format -t $(bin/kafka-storage.sh random-uuid) -c
config/kraft/server.properties
bin/kafka-server-start.sh config/kraft/server.properties
terminal1 :
kafka@ubuntu:~$ java -version
openjdk version "21.0.2"
kafka@ubuntu:~$ cd kafka
kafka@ubuntu:~/kafka$ bin/kafka-storage.sh format -t $(bin/kafka-
storage.sh random-uuid) -c config/kraft/server.properties
Formatting metadata directory /tmp/kraft-combined-logs
kafka@ubuntu:~/kafka$ bin/kafka-server-start.sh
config/kraft/server.properties
[2026-05-25 10:00:00,000] INFO Kafka Server started
terminal 2:
su - kafka
cd ~/kafka
bin/kafka-console-consumer.sh --topic quickstart-events --from-
beginning --bootstrap-server localhost:9092
>Hello Kafka
terminal 3:
kafka@ubuntu:~$ cd ~/kafka
kafka@ubuntu:~/kafka$ bin/kafka-console-consumer.sh --topic
quickstart-events --from-beginning --bootstrap-server localhost:9092
Hello Kafka
```